ChurnSense AI

Predict Churn. Understand Why. Drive Retention.

An Explainable AI (XAI) machine-learning system that predicts customer churn and explains the risk drivers, served through an interactive web app and a REST API — with a second mode that trains a fresh model on *any* uploaded dataset.

Aditi Chougule · github.com/aditichougule/churnsense-ai

Version 2.0 · November 2026

Contents

1. Executive Summary
2. Case Study: From a Broken Model to ROC-AUC 0.85
3. How It Works, Step by Step
    3.1 Data loading and the synthetic-data guardrail
    3.2 Cleaning
    3.3 Feature engineering
    3.4 The modeling pipeline (fit-once / serve-once)
    3.5 Training and model selection
    3.6 Evaluation and metrics
    3.7 Key churn drivers
    3.8 Explainability (linear SHAP values)
4. The Two Modes (Demo and Bring-Your-Own-Data)
5. The Scoring API
6. Architecture and Project Structure
7. Deployment
8. Quality: Testing and Continuous Integration
9. Limitations and Roadmap
10. Glossary

1. Executive Summary

Customer Churn Prediction estimates the probability that a telecom customer will cancel their subscription ("churn"), so that retention teams can act *before* the customer leaves. It is a complete, production-style machine-learning system rather than a notebook experiment.

The product offers two ways to use it:

- **Demo mode** — explore a model trained on the well-known Telco Customer Churn dataset: analyse churn patterns, score individual customers, and read an explanation of every prediction.
- **Bring-your-own-data mode** — upload *any* CSV with a binary target; the app detects feature types, trains and compares models, lets you predict, and download the trained model. This turns the project from a demo into a reusable tool.

On the real Telco test set the deployed model reaches a **ROC-AUC of 0.847** and a **recall of 0.80** — it correctly identifies about 80% of the customers who will actually churn. Predictions are served by both a Streamlit web application and a FastAPI service, and every prediction is accompanied by a per-customer explanation.

At a glance	
Problem	Binary classification: will a customer churn? (Yes / No)
Dataset	Telco Customer Churn — 7,043 customers, 19 features, ~26.5% churn
Deployed model	Logistic Regression (selected over Random Forest by cross-validation)
Headline metrics	ROC-AUC 0.847 · Recall 0.797 · F1 0.611 · Precision 0.496
Interfaces	Streamlit web app · FastAPI REST service · Docker image
Quality	21 automated tests · GitHub Actions CI · per-customer explanations

2. Case Study: From a Broken Model to ROC-AUC 0.85

This project began as an **audit** of an existing codebase, and the audit uncovered a system that looked professional but did not work. Three findings stood out:

- **The dataset was synthetic.** The committed CSV was random values from a fallback function, with the churn label statistically independent of every feature. No model can learn from noise.
- **The shipped model was dead.** It predicted "No Churn" for 100% of customers (F1 = 0, ROC-AUC = 0.49 — a coin toss).
- **The app ignored half its inputs.** A train/serve mismatch silently set five of the user's inputs to zero at prediction time.

The refactor replaced the synthetic data with the real dataset (behind a guardrail that rejects synthetic data), and rebuilt the modeling code around a single *fit-once / serve-once* pipeline that eliminates the train/serve mismatch by construction:

Metric (real test set)	Before	After

ROC-AUC	0.49 (random)	0.847
Recall (churners caught)	0.00	0.797
F1-score	0.00	0.611
Customers flagged as churn	0 of 7,043	realistic

The lesson embedded in the product: a model is only as good as its data and the consistency between how it is trained and how it is served.

3. How It Works, Step by Step

This section follows a customer record from raw data to an explained prediction. The same transformation chain is used during training and serving — the central design idea of the system.

3.1 Data loading and the synthetic-data guardrail

The dataset is a required input — the loader never fabricates data. If the file is missing or looks synthetic (placeholder IDs, or no relationship between contract type and churn) it fails loudly instead of silently poisoning training:

```
def load_telco_data(path):
    if not os.path.exists(path):
        raise FileNotFoundError("Download the real Telco dataset ...")
    df = pd.read_csv(path)
    _assert_is_real_telco(df)   # rejects placeholder / signal-free data
    return df
```

3.2 Cleaning

Cleaning removes duplicates, trims stray whitespace from text fields, and converts TotalCharges (which ships as text with 11 blank cells) to a number. Missing values are **not** filled here — imputation is delegated to the pipeline so that training and serving share exactly one strategy.

3.3 Feature engineering

Four derived features add business signal. They are computed row-by-row, so they behave identically on a 7,000-row training set and on a single live customer:

Feature	Meaning
TotalServices	Count of active add-on services (security, backup, streaming, ...)
ContractRisk	Month-to-month = 3, One year = 2, Two year = 1
ChargeRatio	Monthly charges divided by total charges (tenure-adjusted spend)
TenureSegment	Tenure bucketed into 0-1y, 1-2y, 2-3y, 3y+

3.4 The modeling pipeline (fit-once / serve-once)

Feature engineering, missing-value imputation, scaling of numbers, and one-hot encoding of categories all live inside a single scikit-learn Pipeline object:

```
FeatureEngineer
   -> ColumnTransformer(
        numeric:     impute(median) + StandardScaler
        categorical: impute(most_frequent) + OneHotEncoder(ignore unknown))
   -> Classifier (Logistic Regression / Random Forest)
```

This object is fitted once during training and saved as one file (churn_pipeline.pkl). Both the app and the API call pipeline.predict_proba(raw_dataframe). Because nothing is re-fitted at prediction time, training and serving transform inputs identically — the bug that plagued the original code cannot recur. Unknown categories are handled safely, so single-row and previously unseen values never crash the model.

3.5 Training and model selection

The training script splits off a 20% test set, then selects the best model using **5-fold cross-validation on the training set only**. The test set is touched exactly once, for the final report, so the reported numbers are not optimistically biased. Both candidates use class weighting to handle the ~26.5% churn imbalance.

Candidate	Outcome
Logistic Regression	Chosen — higher CV ROC-AUC (0.851); linear and interpretable
Random Forest	Runner-up — CV ROC-AUC 0.845

3.6 Evaluation and metrics

The deployed model is evaluated on the held-out test set of 1,409 customers. Recall is weighted heavily: in churn, catching customers who will actually leave usually matters more than occasionally flagging a loyal one.

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression (deployed)	0.731	0.496	0.797	0.611	0.847
Random Forest	0.771	0.553	0.727	0.628	0.840



Figure 1 — Model comparison on the held-out test set.

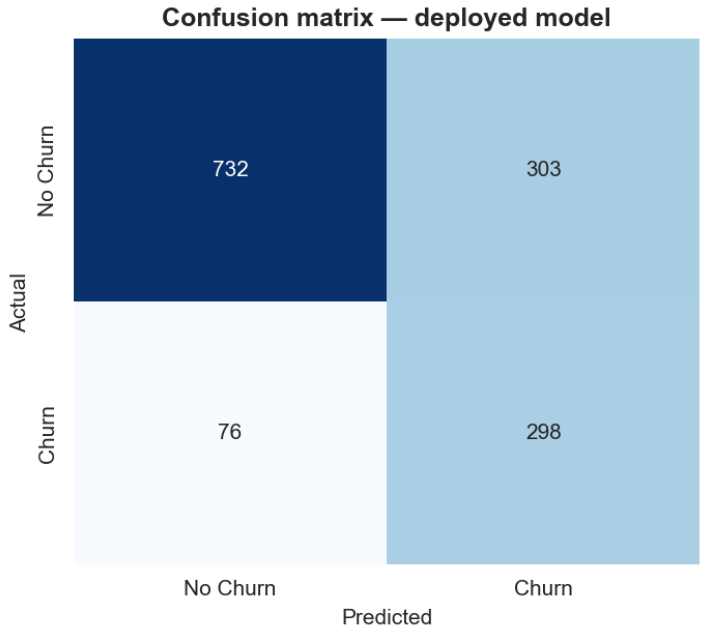


Figure 2 — Confusion matrix of the deployed model; the bottom-right cell is the churners correctly caught.

3.7 Key churn drivers

Permutation importance (how much the score drops when a feature is shuffled) ranks the strongest drivers: total charges and tenure, internet-service type, and contract type. Month-to-month, fibre-optic, newer customers churn far more.

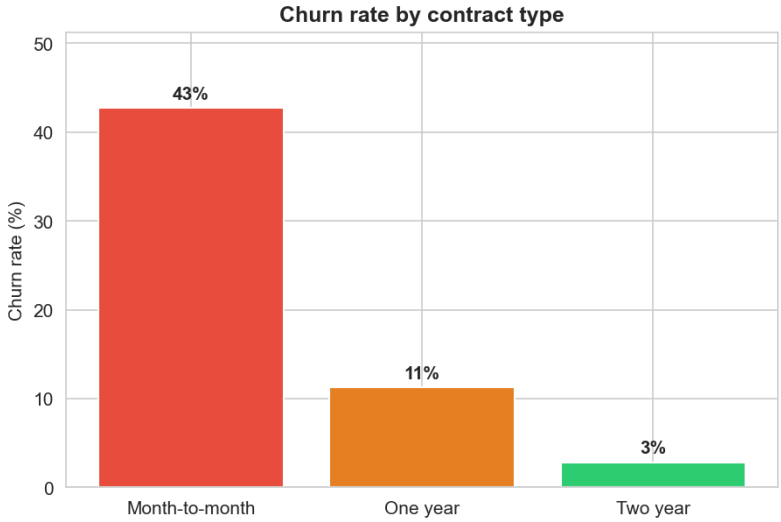


Figure 3 — Churn rate by contract type: 43% for month-to-month vs 3% for two-year.

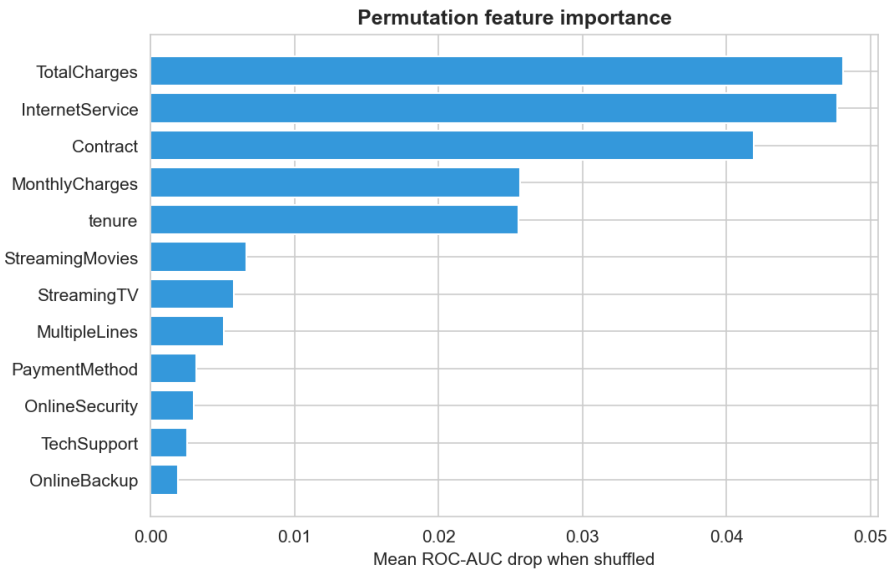


Figure 4 — Permutation feature importance for the deployed model.

3.8 Explainability (linear SHAP values)

Every prediction is explainable per customer. Because the deployed model is linear, the decomposition is **exact**: the churn log-odds equal the model intercept plus the sum of each feature's contribution. These contributions are precisely the (linear) SHAP values, computed with no extra dependencies:

```
logit(churn) = intercept + sum_i ( coefficient_i  x  feature_i )
```

Positive contributions push the customer toward churn (red), negative ones toward retention (green). The same chart appears live in the app under "Why this prediction?".

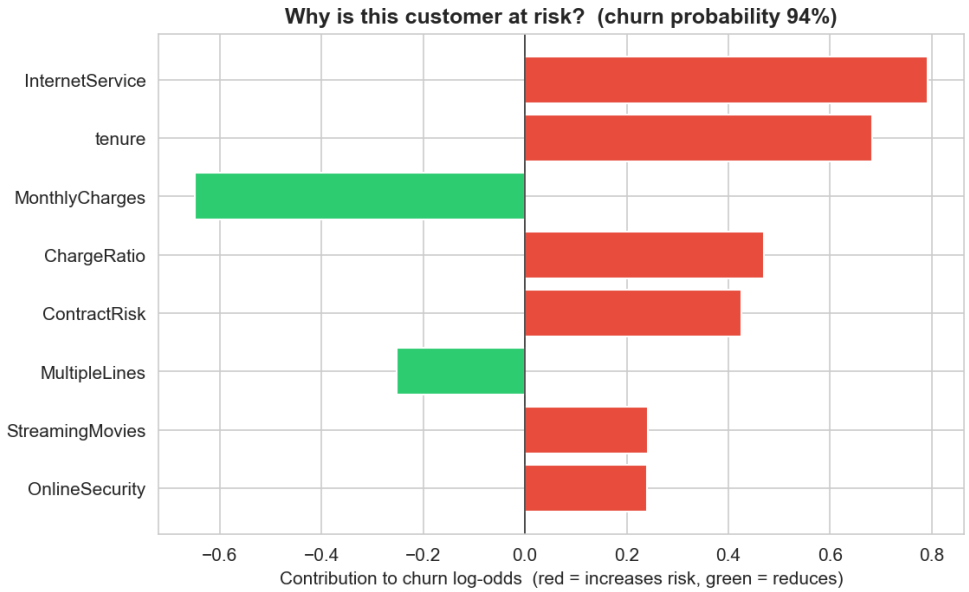


Figure 5 — Per-customer explanation for a high-risk profile (94% churn probability).

4. The Two Modes

4.1 Demo (Telco)

The demo runs on the bundled, pre-trained model and offers five pages:

Page	What it shows
Overview	Dataset size, churn rate, average tenure
Churn Insights	Churn distribution and rates by contract and internet service
Make Prediction	A form to score one customer, with probability, advice and the explanation
Model Details	Live metrics, model comparison and feature importance
Data Overview	The raw dataset, summary statistics and feature distributions

4.2 Bring-Your-Own-Data

The second mode turns the project into a reusable tool. The user uploads a CSV, picks the target column, and the system does the rest:

- **Auto-detect** numeric vs categorical columns; drop ID-like and constant columns.
- **Guardrails** stop bad inputs: minimum rows, a strictly binary target, and a minimum minority-class size — a clear error rather than a silently bad model.
- **Train & compare** Logistic Regression and Random Forest with cross-validation.
- **Inspect** a comparison table, confusion matrix and feature importance.
- **Predict** a single record via an auto-generated form, and **download** the trained pipeline as a portable .pkl (depends only on scikit-learn).

The generic pipeline is as strong as the bespoke one: on the Telco data it reaches a cross-validated ROC-AUC of 0.846, essentially matching the hand-built version.

5. The Scoring API

A FastAPI service exposes the same pipeline over HTTP, so other systems (a CRM, a batch job) can request scores. Requests are validated with Pydantic: invalid categories or out-of-range numbers are rejected automatically with a 422 error.

Method	Path	Purpose
GET	/health	Liveness and which model is loaded
GET	/model	Model metadata (metrics, version, timestamp)
POST	/predict	Score one customer
POST	/predict/batch	Score up to 1,000 customers in one call

Example request and response:

```
POST /predict
{ "gender":"Male", "SeniorCitizen":1, "tenure":1, "Contract":"Month-to-month",
```

```
  "InternetService":"Fiber optic", "MonthlyCharges":95.0, ... }

200 OK
{ "churn": true, "churn_probability": 0.9388, "risk": "high" }
```

6. Architecture and Project Structure

The code is organised so the heavy logic is testable independently of the user interface. The Streamlit app and the API are thin layers over the shared pipeline.

Path	Responsibility
src/pipeline.py	Feature engineering + the single fit-once/serve-once pipeline
src/data_loader.py	Load (with guardrail), clean and validate the dataset
src/explain.py	Per-customer explanations (exact linear SHAP values)
src/automl.py	Generic 'bring your own data' trainer with guardrails
train_model.py	Train, select, evaluate and persist the model + metadata
app.py	Streamlit web app (both modes, all pages)
service/main.py	FastAPI scoring service
tests/	21 automated tests (engine, app flow, API, explanations, AutoML)
Dockerfile, .github/	Container image and continuous integration

7. Deployment

Web app (Streamlit Community Cloud, free). Connect the GitHub repository, choose branch main and file app.py, select Python 3.12+, and deploy. The app loads the committed model, so no training runs on the server.

API (Docker, any container host).

```
docker build -t churn-api .
docker run -p 8000:8000 churn-api    # POST to http://localhost:8000/predict
```

8. Quality: Testing and Continuous Integration

21 automated tests run on every push and pull request via GitHub Actions. They go beyond "does it run" and assert real behaviour:

- **Performance gate** — fails if test ROC-AUC drops below 0.78 or recall below 0.45, catching a model that has degenerated to the majority class.
- **Train/serve guard** — a high-risk profile must score clearly higher than a low-risk one, catching the historical input-dropping bug.
- **Explanation correctness** — the per-feature contributions must reconstruct the predicted probability exactly.
- **API** — health, prediction shape, validation errors (422) and batch scoring.
- **AutoML** — learns signal, drops ID-like columns, and enforces every guardrail.

Before release the whole system was stress-tested end to end: deterministic training, twelve API edge cases, all app pages and the full upload-train-predict-download flow rendered without error, and the

generic engine handled all-numeric, all-categorical, missing-value and unusual-target datasets.

9. Limitations and Roadmap

Honest limitations, documented in the model card:

- The model is trained on a single point-in-time snapshot; there is no drift monitoring or scheduled retraining yet.
- Precision is around 0.50 — roughly half of flagged customers would not actually churn. Acceptable for retention outreach, but not for punitive actions.
- gender is a feature; before any real deployment it should be checked for disparate impact (and possibly removed).

Planned next steps: a live hosted demo, decision-threshold tuning to a business cost, additional algorithms (XGBoost / LightGBM), a fairness check, and model registry / drift detection for true production operation.

10. Glossary

Term	Plain-English meaning
Churn	A customer cancelling their subscription.
ROC-AUC	How well the model ranks churners above non-churners (0.5 = random, 1.0 = perfect).
Recall	Of the customers who actually churn, the share the model catches.
Precision	Of the customers the model flags, the share that actually churn.
F1-score	A single balance between precision and recall.
Pipeline	One object that bundles all data transformations with the model.
One-hot encoding	Turning a category (e.g. contract type) into numeric yes/no columns.
SHAP value	Each feature's contribution to a single prediction.
Cross-validation	Rotating train/validation splits to estimate performance without using the test set.